

INFORME TÉCNICO DE EVOLUCIÓN, HARDWARE Y DESPLIEGUE

Subsistema Firmware: Microcontrolador ESP32-S3, Display LED RGB y Sensores

Proyecto de Grado:	Marcador Electrónico para Natación con Matriz LED y Monitoreo IoT	Fecha de Emisión:	Septiembre de 2026
Repositorio:	github.com/LisandraGar/marcador-electronico-para-natacion-circuitpython	Plataforma:	Adafruit MatrixPortal S3 (ESP32-S3) + CircuitPython 10
Herramienta IDE:	Thonny IDE (Software Tonny) / USB CIRCUITPY	Matriz Física:	RGB LED HUB75 de 128x32 píxeles

1. Resumen Ejecutivo y Alcance de las Mejoras en Hardware

El subsistema **Firmware** actúa como el núcleo de cómputo embebido en tiempo real del marcador electrónico. Ejecutado sobre un microcontrolador **Adafruit MatrixPortal S3 (ESP32-S3)** con el runtime **CircuitPython 10**, gobierna directamente la matriz de LEDs RGB mediante protocolo HUB75, lee los sensores ambientales y de reloj RTC, y mantiene el enlace bidireccional con el broker MQTT sobre TLS.

Innovaciones y optimizaciones incorporadas en esta etapa:

- **Corrección de Resolución Física 128x32:** Reconfiguración de la matriz de 128x64 con `tile_rows=2` a 128x32 con `tile_rows=1` y ajuste de interlineado tipográfico (`line_spacing=9`), eliminando la distorsión visual.
- **Sensor Capacitivo Sumergible de Llegada:** Integración de placa de toque en el carril de natación (Pin board.A1) con detección de flanco positivo, anti-rebote y cooldown de 1.5s contra olas para detener el cronómetro automáticamente al instante exacto del contacto.
- **Actuador Acústico (Bocina/Buzzer):** Controlador en Pin board.A2 para emitir la señal acústica de largada/salida de carrera y la doble señal de confirmación de llegada.
- **Centralización de Parada y Publicación MQTT:** Creación de `record_chrono_stop()` para unificar el registro de records y la sincronización con la aplicación web.
- **Entorno y Librerías Modificadas:** Uso documentado de **Thonny IDE** (Tonny) y suite de librerías personalizadas en `lib/utlis/`.
- **Blindaje de Seguridad Pre-Commit:** Implementación de `scripts/check_secrets.py` para evitar la subida accidental de contraseñas de red o MQTT.

2. Herramientas de Desarrollo y Librerías Modificadas

Para el desarrollo y mantenimiento del firmware se estandarizó el uso del software **Thonny IDE** y una arquitectura modular de librerías:

Uso de Thonny IDE (Software Tonny):

Thonny IDE es el entorno especializado para microcontroladores con CircuitPython. Permite la edición directa de archivos en la unidad virtual CIRCUITPY vía USB, monitorización interactiva en tiempo real mediante la consola **REPL** (Read-Eval-Print Loop) y diagnóstico de excepciones en caliente sin necesidad de ciclos lentos de compilación cruzada.

Estructura de Librerías Modificadas y Propietarias en lib/utlis/:

Archivo / Módulo	Tipo	Función Técnica y Modificaciones Realizadas
<code>touch_sensor.py</code>	Nuevo	Controlador de placa capacitiva de llegada (Pin A1). Filtro de rebotes de agua y cooldown.
<code>buzzer.py</code>	Nuevo	Controlador acústico (Pin A2). Señales sonoras de salida (0.35s) y llegada (2x0.15s).

screen.py	Modificado	Configuración HUB75 ajustada nativamente a width=128, height=32, tile_rows=1.
topic_manager.py	Modificado	Función record_chrono_stop() centralizada para guardar en data.txt y publicar por MQTT.
main.py	Modificado	Bucle de tiempo real con interlineado 9px y verificación instantánea del sensor de llegada.
config.py	Modificado	Carga dinámica de pines de hardware (TOUCH_PIN, BUZZER_PIN) desde settings.toml.
check_secrets.py	Nuevo	Hook de pre-commit en Python que analiza el diff de Git e impide subir contraseñas reales.

3. Trazabilidad de Commits en el Firmware

Hash	Fecha	Autor	Mensaje de Commit	Alcance Técnico
8bb5c3d	2026-06-22	LisandraGar	Terminado	Versión base inicial de firmware
9880938	2026-09-19	Angel Arevalo	docs y config: ajustar resolución 128x32 en screen.py y documentacion	Corrección física de display HUB75
3845489	2026-09-19	Angel Arevalo	feat: sensor capacitivo sumergible de llegada y señal acustica de bocina	Integración de placa táctil y bocina
41ded09	2026-09-19	Angel Arevalo	security: implementar pre-commit hook para prevencion de fugas de contraseñas	Seguridad activa en repositorio

4. Matriz Comparativa: Firmware Previo vs. Firmware Posterior

Dimensión	Estado Previo (Commit 8bb5c3d)	Estado Posterior (Optimizado)
Resolución de Display	- Matrix(128, 64, tile_rows=2). - Desajuste con el hardware real de 128x32. - Interlineado 10px con líneas cortadas.	- Matrix(128, 32, tile_rows=1) nativo. - Interlineado calibrado a line_spacing=9. - Caracteres definidos y legibles a distancia.
Detección de Llegada de Nadador	- Inexistente en el firmware. - La parada solo se podía hacer manualmente desde un botón en la interfaz web. - Inexactitud severa por latencia humana.	- Sensor capacitivo sumergible en Pin board.A1. - Detección física instantánea por flanco al contacto del nadador. - Filtro anti-falsos positivos por oleaje (cooldown 1.5s). - Parada automática al milisegundo.
Señalización Acústica	- Inexistente en el circuito. - Ninguna referencia sonora para los nadadores o jueces.	- Bocina/Buzzer conectada en Pin board.A2. - Pitido firme de 0.35s para largada oficial. - Doble pitido de 0.15s para confirmación de llegada.
Arquitectura de Tiempos y Scores	- Código de guardado acoplado exclusivamente al callback de suscripción MQTT. - Imposibilidad de parada física local.	- Módulo record_chrono_stop() desacoplado. - Detiene el cronómetro, actualiza data.txt y emite por MQTT sin importar si se accionó por sensor o web.
Seguridad de Configuración	settings.toml vulnerable a subirse con credenciales de WiFi y broker.	Scanner pre-commit en Python que audita el diff e impide subir contraseñas reales a GitHub.

5. Esquema de Conexiones Físicas y Despliegue con Thonny IDE

A continuación se detalla el conexionado de pines sobre la placa Adafruit MatrixPortal S3 y el procedimiento de carga:

Periférico / Sensor	Pin MatrixPortal S3	Tipo de Señal	Descripción y Propósito
Sensor Capacitivo Sumergible	board.A1 (GPIO 18)	Entrada Digital / Touch	Placa táctil sumergida al final del carril para parada automática.
Bocina / Zumbador Acústico	board.A2 (GPIO 1)	Salida Digital (Active HIGH)	Emite pitido de salida (largada) y doble pitido de llegada.

Reloj en Tiempo Real (RTC)	board.SDA y SCL	Bus I2C Digital	Módulo DS1307 con batería para mantener hora independiente.
Sensor Temperatura Ambiente	board.A0 (GPIO 17)	Entrada Analógica	Sensor LM35 para captar temperatura ambiental en piscina.
Matriz LED RGB 128x32	Conector HUB75 Frontal	Bus Paralelo RGB HUB75	Dos paneles encadenados de 64x32 con alimentación 5V externa.

Procedimiento de Carga y Configuración en Thonny IDE (Tonny):

1. Conecta la placa MatrixPortal S3 a la PC mediante cable USB-C. Aparecerá la unidad CIRCUITPY.
2. Abre **Thonny IDE**. En la esquina inferior derecha, selecciona como intérprete: **CircuitPython (generic)**.
3. Copia las carpetas lib/, fonts/ y los archivos code.py, boot.py y data.txt a la raíz de CIRCUITPY.
4. En la unidad CIRCUITPY, abre o crea settings.toml con las credenciales correspondientes:
 - CIRCUITPY_WIFI_SSID = "Tu_WiFi" y PASSWORD.
 - MQTT_BROKER = "90ee163ce96e47e2ab0300d92f22e9d5.s1.eu.hivemq.cloud".
 - MQTT_PORT = 8883, MQTT_USERNAME = "marcador_esp32".
 - TOUCH_PIN = "A1" y BUZZER_PIN = "A2".
5. Guarda el archivo en Thonny. La placa se reiniciará automáticamente. En la consola **REPL** verás los mensajes:
 Conectando a WiFi... Conectado exitosamente
 Conectando a broker MQTT... Conexión exitosa
 Bocina/Buzzer inicializado en board.A2
 Sensor capacitivo sumergible inicializado en board.A1